

SHNackathon 2026

The Problem

Medical and healthcare trainees are surrounded by information, yet many still struggle to convert knowledge into durable understanding and clinical reasoning. Despite widespread access to question banks, flashcard systems, AI tutors, and simulation tools, challenges persist: passive memorization, cognitive overload, limited feedback on reasoning, and difficulty transferring knowledge into patient care, etc.

This hackathon challenges you to build something meaningfully better.

The Challenge

Prototype a solution that teaches medical students how to select medications for patients with Type 2 Diabetes Mellitus. You will be provided with a large body of information surrounding the different drugs used for the various complications of diabetes as well as the indications for when to use these drugs and side effects of these drugs. This dataset can be found as PDFs in the "Announcement" slack channel. The goal will be to provide a solution (digital, hardware or hybrid) which exclusively uses the information in the dataset and synthesizes it for medical students to apply in situations where they need to determine and apply what the appropriate treatment is for a patient with Type 2 Diabetes Mellitus based on their presenting symptoms, complaints, and long-term management goals. AI-enabled is strongly encouraged.

1. Your solution must clearly enhance learning in a way that goes beyond existing tools.
2. Your work must be grounded in educational/learning theory. Select at least one recognized learning framework and explain how your design operationalizes it. It is not enough to name a theory; you must demonstrate how it actively shapes the structure of your solution. ****Refer to Appendix 1 below****
3. A core component of this challenge is competitive awareness. Before building, you are required to research what already exists. Consider platforms such as UWorld, AMBOSS, Passmedicine, Anki, adaptive learning systems, AI tutors (e.g., large language model-based tools), virtual standardized patient systems, clinical reasoning simulators, gamified quiz platforms, flashcard systems, and diagnostic decision tools. Then clearly articulate how your solution is designed to outperform.
4. Finally, you must demonstrate your tool in action. During your presentation, use your solution to teach the judges using your educational tool. By the end of your demonstration, the judges should understand the topic more clearly than before. You may choose any of the topics/domains listed in the 'Data Set' file related to T2DM. Aim for your explanation to be clinically accurate and effectively delivered through your platform.

The Deliverables

All teams must submit the following:

1. Devpost Submission

Your Devpost submission must clearly explain your solution and allow the judges to meaningfully evaluate it.

- If your project is a website, provide a working live link.
- If it is an app, it must be deployed in a usable format.
- If it is hardware, include schematics and/or a short video demonstrating a functional prototype.
- You may also include a short video walkthrough showing how your solution works and how it teaches the selected concept.

The goal is clarity and usability. Judges must be able to understand and experience your solution.

2. Pitch Deck (PowerPoint format)

You must submit a PPT file by **2:00 PM (Eastern Time) on Thursday**.

Recommended slide structure:

1. Title
2. Team introduction
3. The problem
4. Your solution
5. Explanation of the learning theory used
6. Technology stack / explanation of your technical approach
7. A clear 3–5 step pipeline showing how your system works
8. Demonstration
9. Impact and future directions

The top 6 projects will be announced by Thursday 6:00PM (Eastern Time) and selected to present **live on Friday**. Teams selected to present may revise and refine their PPT after Thursday's submission. Final presentations must be **10 minutes maximum**.

The prize winners (Top 3 and special prizes will be announced at the end of the Friday live presentations.

What We Are Evaluating

Projects will be assessed on:

- the originality of the idea
- the depth of educational theory integration
- the clinical clarity of the teaching demonstration

- the degree to which the team meaningfully differentiates from existing tools
- and the overall effectiveness of the presentation in helping the judges learn

We are not looking for another quiz app or a simple chatbot interface. We are looking for thoughtful systems that improve reasoning, retention, feedback, and transfer to clinical performance.

The Standardized Dataset

All teams will receive the same standardized clinical dataset at the start of the hackathon. This dataset will contain a curated corpus of clinical, educational, and population health documents related to ****Type 2 Diabetes Mellitus (T2DM)****.

The dataset is your raw material. Your job is to transform it into a learning experience. How you restructure, sequence, and present this material — and what you ask the learner to *do* with it — is the core of the challenge.

The dataset will be released on **Tuesday, Feb 17** via **our Slack channels and Devpost resource link**. You may use one or multiple PDFs of the 'Data set'.

Technical Resources

Platforms and Tools

You may use any tools you choose. Here are recommended starting points:

Development: - [Replit](#) — browser-based IDE, easy deployment - [Claude Code](#) — AI-assisted development (agentic coding) - [Cursor](#) — AI-integrated code editor - [Google AI Studio](#) — Gemini API access - [Vercel](#) / [Netlify](#) — free hosting and deployment

AI APIs: - OpenAI API, Anthropic API, Google Gemini API - [SPECIFY: any API credits or keys being provided to participants]

Design: - Figma (free for students) - Canva

No-code / Low-code: - Replit Agent, Lovable, Bolt, v0

Rules and Policies

What You Can Use

- AI tools for development are **explicitly encouraged**. You may use Claude Code, Cursor, ChatGPT, Copilot, Replit Agent, or any AI-assisted development tool. This is a feature, not a loophole.
- Open-source libraries, frameworks, and APIs are permitted.
- Pre-existing personal code or boilerplate templates may be used as a starting point, but must be disclosed in your Devpost submission. The core innovation must be built during the hackathon.

What You Cannot Do

- You may not submit a project that was substantially built before the hackathon.
- You may not use another team's work without attribution.
- You may not submit to multiple hackathons simultaneously with the same project.
- You may not plagiarize educational content, competitive analysis, or learning theory explanations.

Intellectual Property

- Projects will become open-sourced and teams can choose to post onto Github
- Teams retain full ownership of their projects.
- By submitting, you grant SHN and SHNRI permission to showcase your project (name, description, demo video, screenshots) for educational and promotional purposes.

Academic Integrity

- AI-generated code is permitted. AI-generated analysis, writing, and learning theory explanations must be reviewed, understood, and validated by the team. If you cannot explain your own submission, it will be disqualified.
- Judges may ask questions about your design decisions during Q&A. You must be able to defend them.

Code of Conduct

- All participants must treat each other, mentors, and judges with respect.
- Harassment, discrimination, or hostile behavior of any kind will result in immediate disqualification.

What We Are NOT Looking For

Be warned. The following will score poorly:

- **Another quiz app.** Multiple-choice question banks already exist. UWorld exists. AMBOSS exists. If your tool is “questions + answers + explanations,” you have not differentiated.
 - **A chatbot wrapper.** If your entire solution is “we put a nice UI on ChatGPT,” that is not a learning system. That is a skin.
 - **Name-dropped theory.** If your learning theory slide says “we used spaced repetition” but your tool doesn’t actually implement spaced intervals, you will be penalized, not rewarded.
 - **Feature lists without pedagogy.** We do not care how many features your tool has. We care whether it deliberately engineers learning.
 - **Solutions that don’t work.** A broken demo is worse than a simple one that functions. Ship something that runs.
-

What Separates Good from Great

Good projects will:

- Show that learning theory shaped system architecture — not the reverse
- Demonstrate clear, intentional design decisions
- Accurately describe the cognitive mechanisms being engaged
- Teach the judges something effectively during the live demo
- Function reliably during the presentation

Great projects will:

- Make the judges say, “I wish I had this when I was learning”
 - Demonstrate genuine competitive superiority over existing tools
 - Show evidence of user testing during the hackathon (even informal — teammates as test learners)
 - Present a realistic path to adoption or further development
-

Example Project Directions

These are meant to spark thinking, not constrain it. The best projects will likely be something none of us have imagined yet.

- **Adaptive case-based reasoning trainer** — presents clinical vignettes with progressive disclosure, requires the learner to commit to decisions before revealing next steps,

adapts difficulty based on performance. Grounded in deliberate practice and desirable difficulties.

- **Diagnostic schema builder** — takes the standardized dataset and helps learners build their own illness scripts by organizing findings into patterns, then tests them with interleaved cases that force discrimination between similar diagnoses. Grounded in script theory and interleaving.
- **Metacognitive calibration tool** — asks learners to rate their confidence before answering, tracks calibration over time, and surfaces the gap between what they think they know and what they actually know. Grounded in metacognitive monitoring and the illusion of competence.
- **Multimodal explanation generator** — requires learners to explain concepts in multiple modalities (diagram, written explanation, verbal recording) and evaluates consistency across modalities. Grounded in dual coding theory.
- **AI-generated clinical reasoning coach** — an agent that doesn't give answers but asks Socratic questions, forces the learner to articulate their reasoning chain, and identifies logical gaps. Grounded in elaborative interrogation and retrieval practice.
- **Collaborative diagnostic challenge** — multiplayer system where learners work through cases together asynchronously, seeing each other's reasoning and learning from disagreements. Grounded in social constructivism and collaborative learning.

Appendix 1

How to Demonstrate Learning Theory in Your Design

You are required to ground your solution in at least one established learning theory.

You must demonstrate how your selected theory shapes the way you transform the standardized dataset into a learning experience.

Step 1: Choose a Learning Theory

Select at least one recognized learning framework

Some examples:

- Cognitive Load Theory
- Retrieval Practice / Testing Effect
- Spaced Repetition
- Interleaving

- Deliberate Practice
- Dual Coding Theory
- Transfer-Appropriate Processing
- Script Theory (clinical reasoning)
- Metacognitive Monitoring / Illusion of Competence

Be prepared to briefly explain it in plain language.

Step 2: Show How Theory Shapes Your Design

Your submission must clearly describe four things:

A. How You Restructure the Content

You will receive the same standardized dataset as every other team. Your job is to transform it.

Common terms to describe common tactics:

- *Segmentation into cases*
Break complex material into smaller, manageable clinical scenarios rather than presenting everything at once.
- *Progressive disclosure*
Reveal information step by step instead of giving all data upfront. For example, symptoms first, then exam findings, then labs.
- *Schema-based clustering*
Organize information around meaningful patterns (e.g., mechanisms, systems, or reasoning pathways) rather than long lists of facts.
- *Concept dependency mapping*
Show which ideas must be understood before others make sense.
- *Case variation or interleaving*
Mix related but distinct cases so learners must actively discriminate between similar diagnoses.

Your explanation should answer:

What did you change about the structure of the content, and why?

B. How Learners Interact with It

Describe what the learner is required to do.

Examples:

- Retrieve an answer before seeing feedback
- Commit to a diagnosis before seeing labs/next reveal
- Explain their reasoning in writing
- Rate their confidence
- Compare two similar cases side by side

Be specific. Avoid phrases like “interactive platform.”

Explain the actual behavior the learner performs.

C. What Cognitive Processes Are Being Engaged

Now connect your design to how the brain learns.

For example:

- Are you reducing cognitive overload?
- Are you forcing retrieval instead of passive reading?
- Are you strengthening pattern recognition?
- Are you improving metacognitive awareness?
- Are you promoting transfer to new cases?

D. Why This Should Improve Learning

Make a clear, causal claim.

Not:

But:

Explain why your design should lead to better understanding, retention, reasoning, or transfer compared to standard question banks or AI summarizers.

Required Slide

Your pitch deck must include one visual slide using this structure:

Learning Theory → Design Feature → Learner Behavior → Cognitive Process → Expected Outcome

Example:

Cognitive Load Theory

- Progressive case segmentation
- Focused attention on one variable at a time
- Reduced extraneous load
- Improved schema formation

This slide must be clear and readable.

What We Are Looking For

Strong projects will:

- Show that theory shaped system architecture
- Demonstrate clear, intentional design decisions
- Accurately describe the cognitive mechanisms involved
- Teach the judges something effectively during the live demo

Extremely strong projects will include all the above, plus:

- A protocol for testing their tool in a valid methodology (protoplasm.replit.app)
- Power calculations, using conservative estimates
- REB form completed (REBECK.replit.app)
- Literature review (<https://pub-med-ai-joshualandy.replit.app>)
- Manuscript draft (you're on your own)

Weak projects will:

- Name-drop theories without implementing them
- Present generic quiz interfaces
- Fail to explain how their design changes learner thinking

You are not required to prove long-term outcomes during the hackathon.

You are required to design deliberately.

The question is not: “Is this tool interesting?”

The question is: “Does this tool intentionally engineer learning?”

Good luck, teams!